



AIM 2627 Python Coursework

2026 年 10 月 4 日

目录

0 这份题面怎么读	2
Q1 机器人自检 (10 分)	4
Q2 战斗日志分析 (15 分)	5
Q3 SentryGrid (15 分)	7
Q4 贪心导航 (15 分)	8
Q5 哨兵决策机 (20 分)	9
Q6 巡逻任务 (20 分)	11
Q7 Debug: 昨天还能跑 (5 分)	13
B Bonus: BFS 与排行榜 (+20 分)	14
提交与验收	15

国庆 Python 特训 (Lesson 1-3) 之后的平时考核。你将为一台哨兵机器人编写 **感知** → **载体** → **导航** → **决策** → **任务** → **维护** 全套控制模块。纯 Python 标准库，零硬件依赖，CI 自动验收。

本文件是全部题目的唯一规格说明：任务、规范、评分与提交规则都在这里。环境搭建与本地命令见仓库 README.md (只讲怎么跑起来)，代码骨架 src/main/__init__.py 的 TODO 注释是各题规范的浓缩版。

0 这份题面怎么读

- 铜金色框内是**故事与背景**，可以跳过（虽然我不建议，我辛辛苦苦写的）；**每题正文给出任务与接口，全部规范以分条列表给出，均为验收依据**。
- 规范是唯一的验收依据**。可见测试 (src/tests/) 覆盖大约六成常规行为，本地全绿不等于满分。
- 规范描述主行为与接口契约；个别边界条件与异常输入的精确归属 **未逐一披露**，由自动测试判定。实现时对每条边界做出保守而自治的选择——答疑时段可以求证。
- 代码骨架中每条 TODO 注释都标注了对应的题目与规范主题，方便你在正文里定位。

题目总览

#	题目	对应课	难度	分值	发布
Q1	机器人自检	L1	★	10	主体发布
Q2	战斗日志分析	L2	★★	15	主体发布
Q3	SentryGrid	L3	★★	15	主体发布
Q4	贪心导航	L2/L3	★★	15	主体发布
Q5	哨兵决策机	L3	★★★	20	主体发布
Q6	巡逻任务	综合	★★★	20	主体发布
Q7	Debug：昨天还能跑	全部	★★★	5	主体发布
B	Bonus：BFS + 排行榜	—	★★★★	+20	与主体同步

你的工作沿哨兵软件栈分层推进：

感知 (Q1-Q2) → 载体 (Q3) → 导航 (Q4) → 决策 (Q5) → 任务 (Q6) → 维护 (Q7 Debug)

仿真世界约定

- 坐标 (x, y) ： x 向右增长、 y 向上增长（数学系）；
- 地图符号：障碍物 ■、敌人 ▲、你的哨兵 ●、走过的足迹 ·；
- python main.py 可以看到 ASCII 演示（这并不是测试的一部分，只是用于让你能看到你的机器人是怎么动的；如果你全部用的是标准接口，他应该还挺好看的）。

2026.3 上海

RMUL2026 上海站在上海工程技术大学（在这之前我都不知道这有一个大学）拉开了帷幕，按照惯例我们提前两天到了赛场开始进行一系列的调试。

备场很吵、酒店很烂、羽毛球馆很远、天气很差。

.....

“关于哨兵，他死了。”

.....

“对不起，明年我一定会写出更稳的代码。”

在 2027 年的 RMUL 赛场上，可怜的 AIMer 在经历了过去一年哨兵从未能够正常在赛场上移动的痛后，决定扬了所有哨兵上的代码重写。

哨兵机器人作为赛场中最富有技术含量的机器人，承担着全自动打击的重任。其配备有高性能小陀螺底盘、前向激光雷达和高速工业相机，发射 17mm 小弹丸，但不允许在比赛过程中进行遥控。

为了让哨兵真正能实现全自动打击，我们需要设计一套完整的行为树逻辑，用于告诉机器人应该在什么时候出现在什么位置并做什么操作。

显然，为了知道机器人什么时候在哪出现，我们会需要让机器人能够感知到这个世界。

Q1 机器人自检 (10 分)

Q1 · 背景

如果大家有在 Hello Festival 感受过 AIM 的机器人操作体验,想必会对我们所有机器人在开机自启时很吵闹的一声动静印象深刻。那是我们用于确认机器人通过开机自检,所有功能正常运行所有模块正常启动的标志。错误的开机自启状态包括但不限于机器人一声不吭或者机器人开始发疯并随机攻击附近的所有人。

为了让可怜的 AIMer 不需要以身涉险地物理接触机器人就可以获取相关的状态,我们打算在机器人上电自检后推送一条通知到该收到的人的设备上。

本题实现两个函数,用于生成上电自检报告。

```
def hp_ratio(hp, max_hp): ...          # 血量百分比, 返回 0-100 的 int
def status_report(name, robot_type, hp, max_hp, battery): ...    # 一行报告 str
```

实现须满足以下规范:

1. 血量百分比为 0-100 的整数。
2. 电量档位分为三档: "OK"、"WARNING"、"LOW", 由截断后的电量与告警阈值比较确定。
3. 报告为单行字符串, 各字段以竖线分隔, 格式模板如下——名称左对齐占 10 字符, 机型居中占 10 字符, 两个百分比各右对齐占 3 字符, 档位接在末尾:

```
"{name:<10}|{robot_type:^10}|HP {百分比:>3}%|BAT {电量:>3}%|{档位}"
```

4. 基准样例:

```
status_report("Sentry-07", "HERO", 65, 100, 75)
# -> "Sentry-07 |   HERO   |HP 65%|BAT 75%|OK"
```

Q2 战斗日志分析 (15 分)

Q2 · 背景

AIMer 们 (和他们的 AI) 的代码风格都非常优秀, 大家养成了习惯不断地用 `self.get_logger()` 和 `rclcpp::info()` 来往机器人的日志中扔垃圾。如果大家关注一下每一次运行的 log, 会发现日志的行数会像细胞分裂一样繁殖。

一个成熟的算法工程师应该具备一眼发现日志中真正和你相关的问题的能力, 但很显然当你面对一个五十万行的日志的时候, 这种能力也已然无济于事。(对的我们在 debug 为什么赛场上哨兵动不了的时候真的去翻了那整场比赛的日志、真的有五十万行、VSCode 搜索都会卡) 五十万行的日志扔个 AI 也并不会给你非常迅速的结果, 更大的可能是快速占满上下文然后提示你开始 Auto Compact。

而更令人感到痛苦的, 莫过于所有人的日志格式不一定一致。你会看到很规范的在日志行最前面输出自己是谁的乖宝宝, 也会看到一些我写的 `[ERROR] Boomed controller` 这种完全不讲道理的输出生, 它连自己是谁都没写, 只说了句它炸了, 然后就没了, 后面什么都查不到。

作为新加入团队的各位, 读懂日志是 First thing first。

本题实现 `analyze_damage_log(lines)`: 解析混合格式的伤害日志, 跳过全部脏行, 返回固定契约的统计字典。

```
def analyze_damage_log(lines): ...  
# 输出: {"total": ..., "by_armor": {...}, "most_hit": ..., "avg": ...}
```

实现须满足以下规范:

1. 识别两种有效行格式。传感器行: 形如 "F:32,L:5,R:12", 各段为“字母: 数值”, F/L/R 对应 front/left/right, 段允许缺失。JSON 行: 形如 '{"armor": "front", "damage": 30, "id": 7}', id 字段可选。三种行格式的字面形状:

```
传感器行  "F:32,L:5,R:12"      # F/L/R -> front/left/right, 段允许缺失  
JSON 行   '{"armor": "front", "damage": 30, "id": 7}'  
脏行      空行 / #注释 / 缺字段 / 字段非法 -- 一律跳过
```

2. 其余行 (空行、# 注释、缺字段、字段非法) 一律按脏行跳过, 不计入任何统计。
3. 解析全程不得抛出异常, 一切无法解析的输入按脏行处理。
4. JSON 行的 armor 必须是三个部位之一; damage 必须是严格正整数。
5. 传感器行的每一段必须为 “字母: 正整数” 格式。
6. 带 id 的 JSON 行按 id 去重: 同一 id 只计第一次出现, 后续重复忽略; 不带 id 的行独立计数; 传感器行无 id 概念。
7. 输出契约固定: total 为有效事件伤害总和; by_armor 按三部位分桶累计, 三个部位键在任何情况下恒存在, 未受击为 0; most_hit 为受击伤害最大的部位, 无有效事件时为 None;

```
{"total": int,                                     # 有效事件伤害总和
```

```
"by_armor": {"front": 0, "left": 0, "right": 0},    # 三个键恒存在
"most_hit": str 或 None,
"avg": float}                                     # round(x, 2); 空日志 0.0
```

Q3 SentryGrid (15 分)

Q3 · 背景

在前面几个题目中有提到哨兵会需要感知整个比赛场地和地图，这种感知也必然会包含机器人本身相对于整个地图所处的位置。我们一般将这种感知分为开环和闭环两种类型，分别对应感知返回的数据能否被其他数据来源进行验证。

一种常见的开环控制逻辑是基于哨兵电机的里程计，即一个很简单的积分 $\int_0^t v(\tau) d\tau$ 。这种方案的优点是代码迁移简单，缺点是不适用于麦克纳姆轮底盘，且当电机空转时无法及时捕捉到。

只不过，我们暂时不需要考虑这种问题。

你将拿到的是一个理想化的仿真地图，不存在任何物理制约，没有空转也没有漂移。这个世界里的哨兵遵守几条简单而固执的物理规则。

骨架已提供构造函数、只读属性与 Facing 枚举。本题补全 SentryGrid 的四个 TODO 方法。

```
class SentryGrid:
    def __init__(self, width, height, obstacles, enemy_pos,
                 start_pos=(0, 0), facing=Facing.UP, fuel=100): ...
    # 已提供 (fuel 即电量, 接口名沿用 fuel)
    @current_pos.setter
    def current_pos(self, value): ... # TODO
    def move_forward(self): ... # TODO
    def turn_left(self) / turn_right(self): ... # TODO
```

实现须满足以下规范：

1. current_pos 的赋值做输入校验：只接受长度为 2 的 tuple 或 list，其余类型抛出 TypeError；元素规范化后以 tuple 存储。实现 setter 后，构造函数可改写为 self.current_pos = start_pos，让初始位置复用同一套校验。
2. move_forward：前方格为障碍，判定为碰撞——位置不变、朝向不变、collision_count 加 1；前方可通行则移动到该格。返回执行后的当前位置。
3. 前进消耗 1 单位电量。
4. 电量 ≤ 0 时底盘断电：前进尝试不再产生位移。
5. turn_left / turn_right：原地旋转 90° ，返回新的 Facing。
6. enemy_pos 不得位于障碍物之上（构造时已校验）。

Q4 贪心导航 (15 分)

Q4 ·背景

贪心思想是算法设计中的一个重要思想，其 definition 请参考 [OI-wiki](#)。

贪心导航的逻辑非常符合直觉：哨兵每次只迈一步，在四个相邻格子里挑一个没有障碍、且让到目标的曼哈顿距离严格变小的方向。只要每一步都在变近，总会到的，对吧？

不对。

凹形障碍会告诉你什么叫局部最优：当每一步都无法更近的时候，贪心就死了。它不会绕路——绕路在它眼里是“变远”，而变远是不可接受的。它宁可站在死角里，也不肯先退一步。

如果你不知道我在说什么，可以去抖音搜一下贪心路径规划，应该会给你一些启发。

你的任务是把这条“看起来最聪明”的策略写出来。

本题实现 `next_step_toward`，返回下一步应朝向的 `Facing`。考察曼哈顿距离、条件分支与平局打破规则。

```
def next_step_toward(pos, target, obstacles, current_facing=Facing.UP): ...
```

实现须满足以下规范：

1. 候选方向 = 四邻域中同时满足两个条件的方向：相邻格不是障碍；移动到该格后，到目标的曼哈顿距离相对当前位置**严格减小**。持平不构成候选。
2. 存在多个候选时，先走与目标绝对坐标差较大的轴。
3. 不存在任何候选时，返回 `current_facing`。
4. 本函数不感知地图边界；朝界外方向的判定由 `SentryGrid.move_forward` 处理。
5. 已知局限：凹形障碍死角中，四邻域可能不存在任何严格减距方向，贪心必然失速。这不是实现缺陷；解决方案见 Q6。

Q5 哨兵决策机 (20 分)

Q5 · 背景

RoboMaster 的赛场上，每台机器人背后都坐着一位操作手：步兵靠手速，英雄靠意识，飞镖手靠心态。只有哨兵背后没有人。哨兵的操作手，是你提前写下的每一行 if-else（实际上是很多 xml 文件，但是为了方便理解，这不重要）。

于是操作手的本能必须被翻译成规则。血量被磨到三成，跑不跑？撤出交火区之后，什么时候算安全？视觉说看到敌人了，信不信——单帧的检测会不会只是噪声？热量快爆了，这一发还开不开火？这些问题人类操作手靠肌肉记忆和肾上腺素回答，哨兵只能靠一张提前写好的规则表：一条一条，按序求值，首条命中即返回，永不例外。

同时，我们有一系列规则条款。这些规则条款由一大群坐在 DJI 天空之城里的人们拍脑袋拍出来，然后经过一些看起来不赖的研讨会，最终形成依托答辩。

然而我们必须遵守它。不遵守的后果很简单，裁判系统强制整车断电。

当然，你可以把这些条款拿去问 AI。它大概率会出于好意，帮你把它们“修正”成符合常识的样子。

本题实现纯函数 `decide(sensor, state, hp, heat)`：读入感知数据、当前状态、血量与热量，返回二元组 `(action, new_state)`——其中 `action` 为动作字符串，`new_state` 为 `SentryState` 成员。`SentryState` 枚举已在骨架中提供。

```
class SentryState(Enum):    # 已提供
    PATROL, SUSPECT, ENGAGE, RETREAT, RETURN

def decide(sensor, state, hp, heat): ...
```

输入处理须满足以下规范：

1. `sensor` 字段：`enemy_frames` 为最近几帧的敌检历史（布尔序列，末位为当前帧，长度 1-6，接受 tuple 或 list，元素按真值解释）；`enemy_dist` 为当前敌距（整数或 None）；`robot_type` 为机型标识（"INFANTRY" 或 "HERO"）；`max_hp` 为满血量。字面形状：

```
sensor = {
    "enemy_frames": (True, False, True),    # 末位 = 当前帧，长度 1-6
    "enemy_dist": 3,                        # int 或 None
    "robot_type": "INFANTRY",               # 或 "HERO"
    "max_hp": 120,
}
```

2. `hp_pct` 为当前血量百分比（0-100 的整数）。

规则表中的术语按下述定义计算，均基于规范化后的输入：

- **可见**：`enemy_frames` 末位（当前帧）为真；
- **敌距**：规范化后的 `enemy_dist`；

- **机型**：规范化后的 `robot_type`。

决策规则按 R1 至 R7 的固定顺序求值，首条命中即返回，无例外路径：

1. **R1 (保命优先)**： `hp_pct <= 30` 时返回 ("RETREAT", RETREAT)。该规则优先于其余全部规则，包括贴脸交火。
2. **R2 (撤退保持)**： `state == RETREAT` 时：恢复到安全血量后返回 ("RETURN", RETURN) 转出撤退；否则保持 ("RETREAT", RETREAT)。
3. **R3 (返航单帧)**： `state == RETURN` 时返回 ("MOVE_BASE", PATROL)，与可见性、敌距、热量均无关。RETURN 为单帧过渡状态。
4. **R4 (交火决策)**： `state == ENGAGE` 且可见时：敌距 ≤ 3 返回 ("SHOOT", ENGAGE)；敌距 > 3 时，机型为 HERO 返回 ("MOVE_RIGHT", ENGAGE)，机型为步兵返回 ("MOVE_LEFT", ENGAGE)。
5. **R5 (交火保持)**： `state == ENGAGE` 且不可见时：短暂丢失保持 ("HOLD_FIRE", ENGAGE)；持续丢失返回 ("SCAN", SUSPECT)。
6. **R6 (敌情确认)**： `state` 为 PATROL 或 SUSPECT 且可见时：若 `enemy_frames` 长度 ≥ 2 且末两位（当前帧与其前一帧）均为真，则敌距 ≤ 3 返回 ("SHOOT", ENGAGE)，敌距 > 3 时机型为 HERO 返回 ("MOVE_RIGHT", ENGAGE)、机型为步兵返回 ("MOVE_LEFT", ENGAGE)——与 R4 的判定完全一致；否则（可见但未末两位不全为真）返回 ("SCAN", SUSPECT)。
7. **R7 (默认行为)**： `state` 为 PATROL 或 SUSPECT 且不可见时，PATROL 状态返回 ("PATROL_MOVE", PATROL)，SUSPECT 状态返回 ("SCAN", SUSPECT)。

顺序求值保证规则的完备性与互斥性：R1 拦截全部 `hp_pct <= 30` 的输入，因此 R2–R7 只在 `hp_pct > 30` 时到达；R2 拦截 RETREAT、R3 拦截 RETURN、R4/R5 按可见性拦截 ENGAGE、R6/R7 按可见性拦截 PATROL 与 SUSPECT。任一合法输入恰好命中一条规则，返回值完全确定。

非法输入处理：以下输入在本规范的输入契约之外：`enemy_frames` 为空序列或长度超过 6；`state` 不是 `SentryState` 的五个成员之一；`sensor` 缺少任一字段。对契约外的输入，一律 `raise ValueError`（异常消息内容不作规定），不得静默返回任何结果。demo 同时违反多项契约（空帧历史、非法 `state`），预期抛出 `ValueError`：

```
decide({"enemy_frames": (), "enemy_dist": "far", "robot_type": 3, "max_hp": 0},
       "PATROL", 50, 999) # -> ValueError
```

注意与契约内的防御式规范化区分：字段存在但取值非法（敌距为 `None`/负数/非整数、机型未知、`max_hp` 非正整数）不抛异常，按输入处理规范第 2–4 条规范化。

Q6 巡逻任务 (20 分)

Q6 ·背景

到这里，你写的每一个模块单元测试都是绿的（应该）。接下来，让我们尝试把感知、载体、导航、决策拼成一台完整的哨兵，扔进一张真实的地图，按下运行。

不出意外的话，你的第一轮测试应该会爆红。

这就是集成。每个零件都对，机器不动。

Q4 埋下的问题在此刻爆发：贪心走进了每一步都无法更近的死角，而贪心宁可站着不动也不肯绕路。解决方案其实很古老——走迷宫的时候，用手贴着一侧的墙走：只要迷宫是连通的，就总能绕出去。这就是左手规则。让哨兵在贪心失灵时切入“沿墙走”的脱困模式，绕出死角、距离重新可缩短时再切回贪心——这是你能想到的最简单的脱困策略，也刚好够用。

我把题目出完了以后在一段没有 WiFi 的东航上写了一遍，纯贪心成功率 44%；加上沿墙脱困变成 48%——然后发现沿墙的转向顺序写反了；修好顺序到 80%，加上绕圈超时换手到 97.5%，步数比又超标了；最后调参到 94%，200 张地图稳过。

这样就够了。

你不需要复现我们踩过的每一个坑，但你一个都不踩上去会对不起我浪费的两个小时，所以这道题给你的是 200 张固定地图和 90% 的成功率阈值，而不是一句“能到就算过”。

```
python tools/run_seeds.py --q6 --seed <N> --render # 会是你这两周最好的老师
```

本题实现 `run_patrol` 与 `report_to_json`，把 Q3 的载体与 Q4 的导航组装成完整任务循环。

```
def run_patrol(grid, max_steps=500): ...
# 循环 sense → decide → act
# 终止：到达目标 / max_steps / 电量耗尽
# 返回 {"steps": ..., "collisions": ..., "visited_count": ...,
#       "found_enemy": ..., "success": ...}

def report_to_json(stats): ...
# 确定性序列化；满足确定性所需的具体参数自行选择
```

实现须满足以下规范：

1. 主循环每轮执行：读取载体状态 → 用 Q4 贪心策略选取方向（贪心失速时切入自定义脱困规则）→ 将朝向对齐到目标方向后调用 `move_forward`。
2. 任务模式总电量按 1000 计；tools 与所有测试均按此构造载体。
3. 终止条件：抵达 `grid.enemy_pos`、步数用尽（`max_steps`，默认 500）、电量耗尽。
4. 脱困策略完全由你设计，题面不提供任何建议；唯一约束是下方第 7 条的统计契约与验收阈值。
5. 返回契约的键与类型固定：

```
{"steps": int, "collisions": int, "visited_count": int,
```

```
"found_enemy": bool, "success": bool} # found_enemy 与 success 同义
```

steps 为执行的动作数；collisions 为碰撞次数。

6. report_to_json 将统计字典序列化为确定性的 JSON 字符串；满足确定性所需的具体序列化参数由你选择，以自动测试验收。
7. 验收阈值 (python tools/run_seeds.py --q6 自测, 200 张固定地图): 成功率 $\geq 92\%$; 平均碰撞 ≤ 1.5 ; 成功案例的平均步数与 BFS 最短路之比 ≤ 1.35 。失败地图逐个渲染复现: python tools/run_seeds.py --q6 --seed N --render。

Q7 Debug: 昨天还能跑 (5 分)

Q7 · 背景

“昨天还能跑”是软件工程里最恐怖的五个字，恐怖之处在于它几乎总是真的：昨天，它确实能跑。

我不想在这里回忆这句话到底发生了多少次。作为算法我只需要确保所有东西在我自己电脑上 work，过去一年的经验告诉我，除了一些很小众的就出现过一次的 CPU affinity 问题，其他所有问题都不是我算法的问题。然而 AIM 的魅力就在这里，你以为你是算法，但是实际上你是电控、还偶尔会需要客串一下运营来运输一车面包人。

CI 只能证明你测过的东西没坏，证明不了你没弄坏没测过的东西。

模块里埋着 6 处 bug。这里 AI 原本写了一段很诡异的题目描述。但是我觉得太蠢了，而且也不一定有人看。

`src/main/legacy_patrol.py` 无 TODO、表面可运行。本题要求修复其全部缺陷并撰写分析，须满足以下规范：

1. 模块内共 6 处缺陷。
2. 行为验收以模块内各函数 docstring 写明的契约为准，隐藏测试逐条核对。
3. 提交日志中存在一次标注为 fix、实际把正确实现改错方向的提交。不得依赖提交历史判断正确性，以 diff 与契约为准。
4. 提交物：修复后的代码。

B Bonus: BFS 与排行榜 (+20 分)

背景

贪心为什么卡死？因为它每一步只看四个方向。BFS 为什么不卡？因为它把整张地图摊在桌上完整看了一遍。这不是两道题的差别，这是局部最优与全局最优的差别，也是导航组入组的第一问。

把 Q6 的贪心导航换成 BFS，重跑 200 张地图：成功率奔着 100% 去，平均步数再砍掉四分之一。天下没有免费的午餐—BFS 每走一步都要把整张地图搜索一遍，而贪心只需要看四个方向。什么时候值得为全局最优付出搜索的代价，是你未来在真实机器人上每天都要回答的问题。

排行榜按（成功率，平均步数，平均碰撞）排名。写上你的名字，明年的新人会看到它，我们也会看到你。如果你做到这里，仍然发现没有任何问题一切都很顺，那这将会是你遇到的第一个问题。

本题实现 `bfs_path_length`，替换 Q6 的贪心导航后重跑 200 张地图。实现须满足以下规范：

```
def bfs_path_length(start, target, obstacles): ...  
# 全局最短路步数
```

1. 返回全局最短路的步数：`start == target` 返回 0；目标不可达返回 -1。
2. `obstacles` 由调用方负责包含地图边界；否则对不可达目标的搜索不会终止。
3. 以 BFS 替换贪心导航，重跑 200 张地图：`python tools/run_seeds.py --bonus`。
4. 排行榜按（成功率，平均步数，平均碰撞）排名。

提交与验收

提交流程（七步）

1. GitHub 页面点 **Use this template** 创建仓库（名字建议 aim-2627-py-cw-<学号>）;
2. `git clone` 到本地，直接在 `main` 分支上开发；
3. 启用 AI 会话归档钩子：`pip install 'agent-session-commit[pre-commit]==0.1.3'` 后在仓库内运行 `agent-session-commit install --pre-commit`，按提示选择你使用的 AI 助手与其会话目录；此后每次 `commit` 会自动把新增会话归档并入该提交（`.agent-sessions/bundles/`）；
4. 实现 TODO，**至少 3 个有意义的增量 commit**（如：读题笔记 → 初版 → 修边界），CI 的 `commit` 粒度检查会写入 Actions Summary 供批改参考；
5. 本地 `python -m pytest` 与 `python tools/run_seeds.py --q6` 自测；
6. `git push origin main`，等 Actions 全绿；
7. 在 Actions 的 **CI Summary** 里找到问卷链接，填写提交（每人一次）。

CI 检查四项

①只允许修改 `src/main/**` 与 `README.md`（`tools`、`src/tests/`、`.github/` 改了直接红）；② `autopep8` 风格（`--diff` 非空即败）；③可见 `pytest`；④ `commit` 粒度软检查（ ≥ 15 个、信息有意义——只做提示不拦车）。

评分

- 总分 100 + Bonus 20，构成如下：**自动测试 60 分**（可见 CI + 隐藏测试 + seed 统计，全自动判分）；**AI 使用透明度 20 分**（依据 pre-commit 钩子归档的 AI 会话评审覆盖度与真实性，见下方 AI 使用口径）；**代码质量 + README 20 分**（人工评审：结构、命名、分解、可读性与设计说明）。
- 完成门槛：Q1-Q2 → 可继续 Linux/Cpp 路线；Q1-Q6 达 60 分 → Python 培训通过。

AI 使用口径

我们不反对使用 AI，鼓励你把它当作正常的生产工具。同时明确声明：课程会通过多种方式**抓取与核对 AI 使用情况**——包括 pre-commit 钩子自动归档的 AI 会话（`.agent-sessions/`，随 `commit` 提交）、仓库内的标记检索与 `git` 历史分析。透明地使用 AI 不影响任何得分；**AI 使用透明度占 20 分**，由归档的覆盖度与真实性评定。需要担心的从来不是“用了 AI”，而是隐瞒或伪造使用记录。规范中的每一条边界都会被自动验收。

2026 年 3 月，上海。

那天其实一切都很正常。检录过了，上电测试过了，裁判系统连上，心跳包一格一格地跳。哨兵的控制器没有任何问题，回来的时候底盘还在转，雷达点云在屏幕边缘刷得很好看。我们几个站在检录区外面，没人觉得今天会有什么不一样。

比赛开始，步兵和英雄立刻冲了出去。哨兵一动不动。

那场比赛的日志真的有五十万行，五分钟的比赛，所有的模块忠实的记录着所有的信息，ros-bag 记录了所有 topic 在每一瞬间的数据，我们就看着 emergency_stop 在比赛的完整过程中不断保持在 1。

能让人释怀的答案并不在那五十万行里。后来我们学会了往更早的地方找：在某一个测试没有覆盖到的边缘环境里，在某一条没人逐字读过的条款里，在某一段没有人能讲清楚的代码里。

2026 年 3 月，上海。比赛开始，哨兵在线，但未收到任何比赛开始信号。

实验室角落那台哨兵还在兢兢业业的忽悠一年又一年的新人重走我们的老路，2526 的贴纸没撕，他也仍然像盖革计数器一样走一路响一路。冬天它会被重新拆开、重新接线，装上一份从零写过的代码。明年三月还会有人提前两天到赛场，在吵到听不清说话的备场里调参数，第二天早上七点在离羽毛球馆最近的永和大王享用一顿温暖的早餐。

2027，希望它动起来。

以及，感谢你看到这里。我写的很辛苦